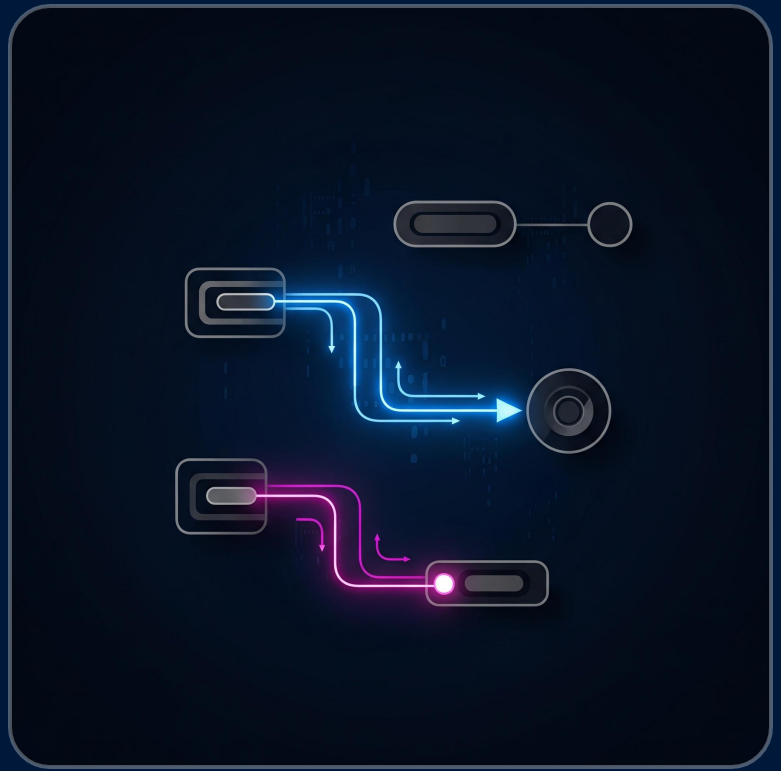
LINEAR DATA STRUCTURES

"Push, pop, walk through nodes, the muscle memory you'll reuse on trees."



▪ TODAY'S ROADMAP ▪

Last class you put Big-O back in your head — you saw the gap between O(n) and O(log n) live in your terminal. Today: a guided tour of the **linear data structures** you already know from ALGO I, but with one eye on what's coming next. Because the way we walk through a linked list today is exactly how we'll walk through a tree in two weeks. Same reflex. One more pointer.

[01] STACK — LIFO

The pile of plates. push, pop, peek — all O(1). Built on a Python list with the END as the top.

[02] QUEUE — FIFO

The canteen line. Use `collections.deque`, never `list.pop(0)`. One looks innocent; it's O(n).

[03] PALINDROME

A stack reverses a string for free. Whenever a problem says "reverse order" → think stack.

[04] LINKED LISTS & THE WALK

The most important pattern of the next 3 weeks. Same skeleton, three data structures.

"Same walking reflex on trees in two weeks. Get it in your fingers today, and trees will feel natural."

STACKS — LIFO

A **stack** is a pile of plates. The last plate you put on top is the first one you take. You never reach to the bottom — that would risk crashing the whole pile. We call this **LIFO**: Last In, First Out.

```
push(v) → put v on top O(1)
pop() → remove and return the top O(1)
peek() → look at the top, no remove O(1)
```

Where stacks show up in real life

Browser back button: Each page visited is pushed. Click "back" → the last visited page pops.

Ctrl+Z (undo): Each action you perform is pushed onto an undo stack. Ctrl+Z pops the most recent.

Python's call stack: Every function call pushes a frame. The function returns → the frame pops. `RecursionError` = too many pushes.

Balanced parentheses: Push every `(` `[` `{`. On a closing bracket, pop and check it matches.

"Whenever a problem says 'reverse order' or 'most recent first' — your reflex should be: stack."

THE PYTHONIC STACK

You don't write a Stack class from scratch — you wrap a Python list. The trick: use the **END** of the list as the top. That single choice makes push, pop, and peek all O(1) for free.

```
class Stack:
def __init__(self):
self._data = [] # underlying storage
def push(self, v):
self._data.append(v) # add at the END → O(1)
def pop(self):
if not self._data: raise IndexError("empty stack")
return self._data.pop() # remove the END → O(1)
def peek(self):
if not self._data: raise IndexError("empty stack")
return self._data[-1] # read the END → O(1)
```

Why END, not START?

01. list.append(v) → O(1)
Adds at the right edge. No shifting needed — the array has spare capacity at the end.

02. list.pop() → O(1)
Removes the LAST element. Same reason: nothing moves.

03. list.pop(0) → O(n)
Removes the FIRST element. Every other element shifts down by one. *Don't use it for stacks.*

"The end of a Python list is the top of your stack. End is your friend; start is your enemy."

▪ QUEUES — FIFO ▪

A **queue** is a line at the canteen. First arrived, first served. **FIFO**: First In, First Out. In Python the right tool is **`collections.deque`** — never a plain list, even though it would compile.

```
from collections import deque
q = deque()
q.append("Alice") # enqueue at the back
q.append("Bob")
q.append("Charlie")
q.popleft() # → "Alice" (the first served)
```

❌ list as a queue

```
q = []
q.append("Alice")
q.pop(0) # O(n) — shifts ALL
```

Looks fine. Silently O(n). On a million items: half a trillion shifts over its life.

✅ deque — the right tool

```
from collections import deque
q = deque()
q.append("Alice")
q.popleft() # O(1)
```

Doubly-linked. Both ends are O(1) for append and pop.

You'll meet queues again in Class 06 — BFS (Breadth-First Search) walks a tree level by level using a queue. Keep `` `from collections import deque` `` in muscle memory.

"`list.pop(0)` looks innocent. It's not. Reach for `deque` every single time you need a queue."

A STACK IN ACTION — PALINDROME

A **palindrome** is a string that reads the same forwards and backwards: `"racecar"`, `"level"`, `"madam"`.
How would you check one? You could use two pointers comparing characters. Or, more interesting today, you could **let a stack do the reversing for you**.

```
def is_palindrome(s):
stack = Stack()
for c in s: # 1. push every char
   stack.push(c)
reversed_chars = []
while not stack.is_empty(): # 2. pop them all
   reversed_chars.append(stack.pop())
return s == "".join(reversed_chars)

is_palindrome("racecar") # → True
is_palindrome("hello") # → False
```

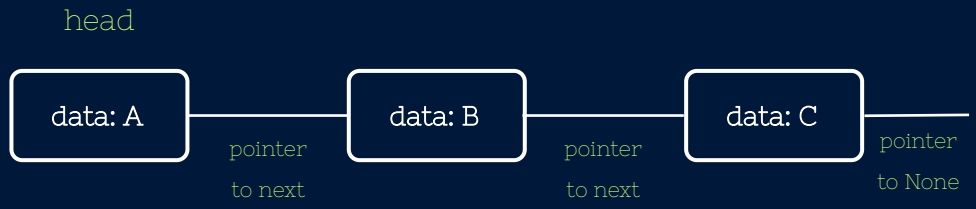
Notice what just happened: we never wrote any reversing logic. The **LIFO property of the stack reversed the string for us, for free**.

We pushed in order A→B→C, popped in order C→B→A. That's the whole trick.

"Whenever a problem says 'reverse order' — your reflex is: stack. The structure does the work for you."

▪ LINKED LISTS — A CHAIN OF NODES ▪

A **linked list** is a chain of nodes. Each node holds a piece of data and a **pointer** (.next) to the next node. The list itself is just a reference to the head — the first node. **There is no index.** To reach the 5th element, you must walk past nodes 1, 2, 3, 4 first.



Why study this? For 99% of your daily Python code, you'll just use a regular list. So why bother with linked lists?

Because the way you walk through one is the **EXACT pattern** we'll use on trees, graphs, and recursive structures. The shape changes; the walking reflex stays the same.

python

```python
class _Node: # underscore = "internal use"
    def __init__(self, data, next=None):
        self.data = data
        self.next = next # pointer to next

class SinglyLinkedList:
    def __init__(self):
        self._head = None # chain starts empty
        self._size = 0
```

▪ TRADE-OFFS AT A GLANCE ▪

Four linear structures, four operations, all the complexities you need to memorize this semester. The takeaway: **each structure is the right answer for some problem and the wrong answer for the next one**. Pick by the operation that has to be fast in YOUR scenario.

Operation	Python list	Linked list	Stack (LIFO)	Queue (FIFO)
Access	O(1)	O(n)	n/a	n/a
Add End	O(1)*	O(n)	O(1)	O(1)
Add Start	O(n)	O(1)	n/a	O(1)
Remove	O(1)/O(n)	O(1)/O(n)	O(1)	O(1)
Memory	Contiguous	Scattered	List-based	Deque
Default	`list`	manual	`list`	`deque`

01. Python `list` is the daily driver.
Random access is O(1). Use it 99% of the time.

02. Linked lists win at ends.
Especially the START — list.insert(0) is O(n), linked-list head insert is O(1).

03. Stacks and queues are interfaces.
You implement them with a list (stack) or a deque (queue). Don't reimplement.

"The structure isn't the answer — the operation that must be fast is. Pick accordingly."

THE WALK — MEMORIZE THIS

This is **the most important pattern of the next three weeks**. Type it once. Close your eyes. Type it again. Everything else this semester is just *"what do you do at each step?"*

```
cur = self._head
while cur is not None:
# do something with cur.data
cur = cur.next
```

Same skeleton, three data structures

01. On a linked list
Follow `.next`. Stop when you hit `None`. The skeleton runs unchanged.

02. On a binary tree (Class 06)
Each node has `.left` and `.right`. The walk becomes recursive — visit left, then right.

03. On a graph (later)
Nodes have `.neighbors`. Walk with a `visited` set so you don't loop forever.

The walking reflex stays. The shape of the data changes. That's why we type this pattern today, slowly and carefully.

WALK #1 — ADD _ TAIL: WALK & ATTACH

To add a node at the **end** of a linked list (without a tail pointer), you must walk to the last node FIRST, then attach. The loop condition matters — read it carefully, it's not the same as in `find_max`.

```
def add_tail(self, v):
    new = self._Node(v)
    if self._head is None: # empty list?
        self._head = new # the node IS the head
    else:
        cur = self._head
        while cur.next is not None: # walk to LAST node
            cur = cur.next
        cur.next = new # attach
    self._size += 1
```

The loop condition trap

❌ `while cur is not None:`
cur becomes None after last node; cur.next = new crashes.

✅ `while cur.next is not None:`
Stop AT the last node. cur is alive — we attach to it.

*"Keep walking while there's still something *after* cur. The instant cur.next is None, stop — cur IS the last node."*

Don't forget

Update `self._size += 1`. Every method that mutates the list must keep the size honest.

"Stop AT the last node when you need to attach. Walk PAST it when you need to read."

WALK #2 — FIND _ MAX: WALK & TRACK

To find the maximum value, walk every node and keep a running max. The skeleton is **identical to `add_tail`** — only the action at each step changes. That's the whole lesson of this slide.

```
def find_max(self):
    if self._head is None:
        return None # empty list

    current_max = self._head.data
    cur = self._head.next

    while cur is not None: # visit EVERY node
        if cur.data > current_max:
            current_max = cur.data
        cur = cur.next

    return current_max
```

Feature	`add_tail`	`find_max`
Goal	Attach node	Read values
Loop Condition	`while cur.next is not None`	`while cur is not None`
Stops	AT the last node	AFTER the last
Action	None — we just walk to find it	Compare and update `current_max`

same skeleton, different action: Read both functions side by side. The walking machinery is identical. THE WALK is a frame; you fill it with the right action.

"Same skeleton, different action. That's THE WALK."

THE BRIDGE TO TREES — SAME REFLEX, MORE POINTERS

Each linked-list node has **one** .next. A binary-tree node has **two** pointers: .left and .right. The walking reflex is identical — you just walk both branches at each step. Today's find_max will come back as find_max_value(node) on a tree in Class 06.

```
# LEFT — Linked list walk (today)
cur = self._head
while cur is not None:
    look(cur.data)
    cur = cur.next

*One pointer. Loop while not None.*
```

```
# RIGHT — Tree walk (preview, recursive)
def visit(node):
if node is None:
    return
look(node.data)
visit(node.left)
visit(node.right)

*Two pointers. Same logic. Recursion replaces while.*
```

The recursive version is the same idea: **base case** is "node is None → return" (= the loop's exit condition). **Recursive case** is "look at this node, then walk both children" (= the loop body, just twice).

Today's find_max is find_max_value(node) on a tree next month — same logic, except we walk left AND right at every node. **Get the pattern in your fingers today and trees will feel natural.**

▪ TODAY WE LEARNED — AND WHAT'S NEXT ▪

TODAY WE LEARNED

01. STACK = LIFO: Built on a Python list with the **END as the top**. push, pop, peek all O(1). Use for "reverse order" or "most recent first". (Undo, Call Stack).

02. QUEUE = FIFO: Reach for collections.deque. list.pop(0) is O(n) — a silent performance trap. Coming back for BFS.

03. PALINDROME: LIFO reverses a string for free. Whenever a problem says "reverse order" → think stack.

04. THE WALK: The pattern of the next 3 weeks.

```
cur = self._head
while cur is not None:
  # do something
  cur = cur.next
```

Same skeleton on linked lists, trees, and graphs.

NEXT CLASS

Hash tables — the structure that makes Python's dict magical.

- **Counting things** (word frequency, log lines).
- **Two-sum** — O(n²) becomes O(n) **for free**.

Then we'll plant a small **seed of recursion** to feel the syntax. The deep dive comes after.

HASH TABLES → THE FREE LOOKUP

"Same walk on trees in two weeks. Just two pointers instead of one."